Document number: P0567R0
Date: 2017-01-30
Project: SG1, SG14
Authors: Gordon Brown, Ruyman Reyes, Michael Wong
Emails: gordon@codeplay.com, ruyman@codeplay.com, michael@codeplay.com
Reply to: michael@codeplay.com, gordon@codeplay.com

Asynchronous managed pointer for Heterogeneous computing

Introduction

Summary

This paper proposes an addition to the C++ standard library to facilitate the management of a memory allocation which can exist consistently across the memory region of the host CPU and the memory region(s) of one or more remote devices. This addition is in the form of the class template `managed_ptr`; similar to the `std::shared_ptr` but with the addition that it can share its memory allocation across the memory region of the host CPU and the memory region(s) of one or more remote devices.

Aim

The aim of this paper is to begin an exploratory work into designing a unified interface for data movement. There are many different data flow models to consider when designing such an interface so it is expected that this paper will serve only as a beginning.

The approach proposed in this paper does not include all of the use cases that a complete solution would cover.

- This approach makes the assumption that there is only a single host CPU device which is capable of performing synchronisation with **execution contexts**.
- This approach does not include an optimized manner of moving data between **execution contexts**.
- This approach does not include support for systems which allows for multiple devices to access the same memory concurrently.

Introduction

Motivation

Non-heterogeneous or distributed systems where there is typically a single device; a host CPU with a single memory region; an area of addressable memory. In contrast, heterogeneous and distributed systems have multiple devices (including the host CPU) with their own discrete memory regions.

A device in this respect can be any architecture that exists within a system that is C++ programmable; this can include CPUs, GPUs, APUs, FPGAs, DSPs, NUMA nodes, I/O devices and other forms of accelerators.

This introduces additional complexity for accessing a memory allocation that is not present in current C++, the requirement that said memory allocation be available in one of many memory regions within a system throughout a program's execution. For the purposes of this paper, we will refer to such a memory allocation as a managed memory allocation as it describes a memory allocation that is accessible consistently across multiple memory regions. With this requirement comes the possibility that a memory region on a given device may not have the most recently modified copy of a managed memory allocation, therefore requiring synchronisation to move the data.

As the act of dispatching work to a remote device to be executed was once only a problem for third party APIs and so too was the act of moving data to those devices for computations to be performed on. However, now that C++ is progressing towards a unified interface for execution [1] the act of moving data to remote devices to be accessible for dispatch via executors is now a problem for C++ to solve as well, a unified interface for data movement of such. The act of moving data to a remote device is very tightly coupled with the work being performed on said remote device. This means that this unified interface for data movement must also be tightly coupled with the

unified interface for execution.

Influence

This proposal is influenced by the SYCL specification [2] and the work that was done in defining it. This was largely due to the intention of the SYCL specification to define a standard that was based entirely in C++ and as in line with the direction of C++ as possible. In order to develop this proposal further, we also seek the experience from other programming models including HPX [3], KoKKos [4], Raja [5], and others.

This approach is also heavily influenced by the proposal for a unified interface for execution [1] as the interface proposed in this paper interacts directly with this.

Scope of this Paper

There are some additional important considerations when looking at a container for data movement in heterogeneous and distributed systems, however, in order to prevent this paper from becoming too verbose, these have been left out of the scope of the proposed additions here. It is important to note them in relation to this paper for future works, for further details on these additional considerations see the future work section.

Naming considerations

During the development of this paper, many names were considered both for the `managed_ptr` itself and for its interface.

Alternative names that were considered for `managed_ptr` were `temporal` as it described a container which gave temporal access to a managed memory allocation, `managed_container` as the original design was based on the `std::vector` container and `managed_array` as the managed memory allocation is statically sized.

Alternative names for the `put()` and `get()` interface were `acquire()` and `release()` as you were effectively acquiring and releasing the managed memory allocation and `send()` and `receive()` as you are effectively sending and receiving back the managed memory allocation.

Proposed Additions

Requirements on Execution Context

For the purposes of this paper, it is necessary to introduce requirements on the **execution context**; the object associated with an **executor** which encapsulates the underlying execution resource on which functions are executed on.

The **execution context** must encapsulate both the execution agents and a global memory region that is accessible from those execution agents. It is required that the memory region of the **execution context** be cache coherent across the execution agents created from the same invocation of an execution function. It is not required that the memory region of the **execution context** be cache coherent across execution agents created from different execution functions or on different **execution contexts**.

*Note: some systems are capable of providing cache coherency across execution functions or **execution contexts** through the availability of shared virtual memory addressing or shared physical memory. This feature is discussed in more detail in the future work section.*

Managed Pointer Class Template

The proposed addition to the standard library is the `managed_ptr` class template (Figure 1). The `managed_ptr` is a smart pointer which has ownership of a contiguous managed allocation of memory that is shared between the host CPU and one or more **execution contexts**. It is important to note here that an **execution context** may be on the host CPU accessing the host CPU memory region, however, the interface remains the same.

At any one time the managed memory allocation can exist in the memory regions of the host CPU and any number of **execution contexts**. However, the managed memory allocation may only be accessible in one of these memory regions at any given time. This memory region is said to be the accessible memory region. If the accessible memory region is of the host CPU, the host CPU is said to be accessible and if the accessible memory region is of an **execution context** that **execution context** is said to be the accessible

execution context.

For the host CPU to be accessible or for an **execution context** to be the accessible **execution context**, if it is not already, a synchronisation operation is required. A synchronisation operation is an implementation defined asynchronous operation which moves the data from the currently accessible memory region to another memory region. From the point at which a synchronisation operation is triggered the currently accessible memory region is no longer accessible. Once the synchronisation point is complete the memory region the data is being moved to is now the accessible memory region.

Synchronisation operations are course grained synchronisation in that they synchronise the entire managed memory allocation of a `managed_ptr`.

Note: some systems are capable of providing finer-grained synchronisation via atomic operations, however, this is generally only when there is shared virtual memory addressing or shared physical memory. This feature is discussed in more detail in the future work section.

There are three ways in which a synchronisation operation can be triggered, each of which will be described in more detail further on. The first is by calling a member function or customisation point of an **executor** triggering a synchronisation operation to the memory region of said **execution context**. The second is by calling the `get()` member function on the `managed_ptr` itself as this will call the above-mentioned member functions of the **executor**. The third is by passing a `managed_ptr` to an **executor** control structure such as `async()`, as this will implicitly trigger the above-mentioned member functions on the **executor**.

Any `managed_ptr` can only have a single host CPU that can be accessible for any given application, that is capable of triggering synchronisation operations.

Note: some systems may wish to have multiple host CPU nodes which are capable of triggering synchronisation operations. This feature is discussed in more detail in the future work section.

If the host CPU is not accessible the `managed_ptr` is required to maintain a pointer to the accessible **execution context** in order for it to perform synchronisation operations.

Memory is only required to be allocated where it is accessible. The managed memory allocation is only required to be allocated on the host CPU memory region if the `managed_ptr` is constructed with a pointer or if a synchronisation operation is triggered to the host CPU. managed memory allocation is only required to be allocated on the memory region of an **execution context** if a synchronisation operation is triggered to that **execution context**.

```
namespace std {
namespace experimental {
namespace execution {

/* managed_ptr class template */
template <class T>
class managed_ptr {
public:

  /* aliases */
  using value_type      = T;
  using pointer         = value_type *;
  using const_pointer   = const value_type *;
  using reference       = value_type &;
  using const_reference = const value_type &;
  using future_type     = __future_type__;

  /* constructors */
  managed_ptr(size_t); // (1)
  managed_ptr(pointer, size_t); // (2)
  managed_ptr(const_pointer, size_t); // (3)
  template <typename allocatorT>
  managed_ptr(size_t, allocatorT); // (4)

  /* copy/move constructors/operators, destructor */
  managed_ptr(const managed_ptr &);
  managed_ptr(const managed_ptr &&);
  managed_ptr &operator=(const managed_ptr &);
```

```
  managed_ptr &operator=(const managed_ptr &&);
  ~managed_ptr();

  /* synchronisation member functions */
  bool is_accessible() const;
  future_type get() const;

  /* operators */
  reference operator[](int index);
  const_reference operator[](int index) const;

  /* other member functions */
  size_t size() const;
};
}  // namespace execution
}  // namespace experimental
}  // namespace std
```

Figure 1: managed_ptr class template

The `managed_ptr` class template has a single template parameter; `T` specifying the type of the elements.

The type parameter `T` specifies the element type of the container, any `T` satisfies the `managed_ptr` element type requirements if:

- `T` is a standard layout type.
- `T` is copy constructible.

A `managed_ptr` can be constructed in a number of ways:

- for constructor (1) the `managed_ptr` allocates the number of elements specified by the `size_t` parameter using the default allocator, therefore allocating on the host remote device memory region.
- for constructor (2) the `managed_ptr` takes ownership of the pointer parameter.
- for constructor (3) the `managed_ptr` takes ownership of the const_pointer parameter.
- for constructor (4) the `managed_ptr` allocates the number of elements specified by the `size_t` parameter using the allocator specified by the allocatorT parameter.

The default constructor T is not called for the elements of the `managed_ptr` during allocation.

The destructor is not required to perform any synchronisation operations.

The member function `get()` triggers a synchronisation operation to the host CPU if the host CPU returns a `future_type` object which can be used to wait on the operation completing. If an error occurred during the synchronisation point or the pointer to the accessible **execution context** is no longer valid then an exception is thrown and stored within the `future_type` that is returned. It is undefined to call `get()` within a function executing on an **execution context**.

The member function `is_accessible()` returns a boolean specifying whether the host CPU is accessible.

The subscript operator returns a reference to an element of the pointer at the index specified by `index` parameter. If the subscript operator is called on the host CPU and the host CPU is not accessible then the return value is undefined. If the subscript operator is called within a function executing on an **execution context** and said **execution context** is not the accessible **execution context** then the return value is undefined.

The member function `size` returns the number of elements of type `T` stored within the managed allocation.

Extensions to Executors and Execution Contexts

In order to facilitate the synchronisation operations required to support the **managed_ptr** the following extensions are proposed for the unified interface for execution.

These extensions are in the form of member functions and global customisation point functions which each trigger a synchronisation operation. The `put()` and `then_put()` functions trigger a synchronisation operation to an **execution context**. The `get()` and `then_get()` functions trigger a synchronisation operation to the host CPU. Each function takes a variadic set of `managed_ptr` and triggers a synchronisation operation for each of them and the global customisation point functions take an **executor** parameter. Each

function returns a `future_type` object that can be used to wait on the synchronisation operation. The `then_get()` and `then_put()` functions also take an `future_type` predicate parameter.

```
namespace std {
namespace experimental {
namespace execution {

/* executor classes */
class <executor-class> {
  ...

  template <typename... ManagedPtrTN>
  executor_future_t<<executor-class>, void> put(ManagedPtrTN...);
  template <typename... ManagedPtrTN>
  executor_future_t<<executor-class>, void> get(ManagedPtrTN...);
  template <typename Predicate, typename... ManagedPtrTN>
  executor_future_t<<executor-class>, void> then_put(Predicate pred, ManagedPtrTN...);
  template <typename Predicate, typename... ManagedPtrTN>
  executor_future_t<<executor-class>, void> then_get(Predicate pred, ManagedPtrTN...);

  ...
};

/* customisation points */
template <typename Executor, typename... ManagedPtrTN>
executor_future_t<Executor, void> put(Executor, ManagedPtrTN...);
template <typename Executor, typename... ManagedPtrTN>
executor_future_t<Executor, void> get(Executor, ManagedPtrTN...);
template <typename Predicate, typename Executor, typename... ManagedPtrTN>
executor_future_t<Executor, void> then_put(Predicate pred, Executor, ManagedPtrTN...);
template <typename Predicate, typename Executor, typename... ManagedPtrTN>
executor_future_t<Executor, void> then_get(Predicate pred, Executor, ManagedPtrTN...);

}  // namespace execution
}  // namespace experimental
}  // namespace std
```

Figure 2: Extensions to unified interface for execution

As described previously there are three ways in which a synchronisation operation can be triggered, and these are separated into explicit and implicit.

The first is by calling one of the member functions or global customisation point functions described above. This will trigger a synchronisation operation to the memory region of the **execution context** associated with the **executor**. This method is useful as it allows finer control over the synchronisation operations by chaining continuations.

The second is by calling the `get()` member function on the `managed_ptr` itself as this will implicitly trigger a call to `get()` on the current accessible **execution context** if the host CPU is not currently accessible. This method is useful when you want to synchronise data back to the host CPU in order to use the resulting data in regular sequential code.

The third is by passing a `managed_ptr` to an executor control structure such as `async()`, as this will implicitly trigger a call to the `put()` member function on the **executor**. If the host CPU is not accessible at this point then this will implicitly trigger a call to `get()` on the `managed_ptr` which will subsequently implicitly trigger a call to `get()` on the currently accessible **execution context**. This method is useful as it allows you to simply pass a `managed_ptr` directly to a control structure without requiring an explicit call to `put()`.

Examples

There is a wide range of user cases that affect the design of a potential interface when it comes to asynchronous tasks and the

movement of the data that those tasks require, however for the purposes of this paper the following example should provide a base requirement for the features this paper proposes.

The following examples show how the features that will be presented in this paper can be used to move data from the host CPU memory region to the memory region of a GPU in order to perform an operation on said data via the **executor** interface.

The first example shows how this would look using the explicit interface for synchronisation operations (figure 3) and the second shows how this would look using the implicit interface for synchronisation operations (Figure 4).

```
/* Construct a context and executor for executing work on the GPU */
gpu_execution_context gpuContext;
auto gpuExecutor = gpuContext.executor();

/* Retrieve gpu allocator */
auto gpuAllocator = gpuContext.allocator();

/* Construct a managed_ptr ptraA allocated on the host CPU */
std::experimental::execution::managed_ptr<float> ptrA(1024);

/* Construct a managed_ptr ptrB allocated on the GPU execution context */
std::experimental::execution::managed_ptr<float> ptrB(1024, gpuAllocator);

/* Populate ptrA */
populate(ptrA);

/* Construct a series of compute and data operations */
auto fut =
  std::experimental::execution::put(gpuExecutor, ptrA)
      .then_put(gpuExecutor, ptrB)
          .then_async(gpuExecutor, [=](auto _ptrA, auto _ptrB) { /* ... */ }, ptrA, ptrB)
            .then_get(gpuExecutor, ptrA, ptrB);

/* Wait on the operations to execute */
fut.wait();

/* Print the result */
print(ptrB);
```

Figure 3: Example of explicit interface

```
/* Construct a context and executor for executing work on the GPU */
gpu_execution_context gpuContext;
auto gpuExecutor = gpuContext.executor();

/* Retrieve gpu allocator */
auto gpuAllocator = gpuContext.allocator();

/* Construct a managed_ptr ptraA allocated on the host CPU */
std::experimental::execution::managed_ptr<float> ptrA(1024);

/* Construct a managed_ptr ptrB allocated on the GPU execution context */
std::experimental::execution::managed_ptr<float> ptrB(1024, gpuAllocator);

/* Populate ptrA */
populate(ptrA);

/* Perform the compute operation with implicit synchronisation operations to the GPU execution context */
std::experimental::execution::async(gpuExecutor, [=](auto _ptrA, auto _ptrB) { /* ... */ }, ptrA, ptrB).wait();

/* Perform a synchronisation operation to the host CPU */
```

```
ptrB.get().wait();

/* Print the result */
print(ptrB);
```

Figure 4: Example of implicit interface

Future Work

There are many other considerations to make when looking at a model for data movement for heterogeneous and distributed systems, however, this paper aims to establish a foundation which can be extended to include other paradigms in the future.

More Complex Data Movement Policies

We may wish to introduce more control over the way in which data is moved between devices at a high level and the consistency between execution contexts. This could be done by having static type traits associated with executor classes which describe the data movement and consistency properties of the execution context associated with the executor.

More Complex Execution Ordering Policies

We may wish to introduce more control over the order in which operations (both data movement and compute) are executed. This could be done by having static type traits associated with executor classes which describe the ordering guarantees between operations, so rather than always having a strict sequential ordering, an implementation may want to relax the requirements based on access dependencies to allow for optimisations.

Synchronisation Operations From Multiple Host CPU Devices

We may wish to introduce the ability for a `managed_ptr` to trigger synchronisation operations from multiple host CPU nodes. This could be done by having an additional concept which allows an execution context to be a host node, another device that is capable of triggering synchronisation operations. The `put()` and `get()` in this case would be relative to the current host node. So, for example, a `put()` on node A to node B would be the equivalent to a `get()` on node B from node A.

Additional Containers

We may wish to extend this principle of a managed pointer to other containers that would be useful to share across heterogeneous and distributed systems such as vectors or arrays. This could be done by having containers such as `managed_vector` or `managed_array` that would have similar requirements to the standard containers of the same names in terms of storage and access yet would be extended to support access from remote devices as with the `managed_ptr` .

Data Movement Customisation Points

We may wish to add customisation points to optimise data movement. This could be done by introducing data movement channels, which can be implemented to optimise data movement between specific execution contexts or between an input or output stream. These could be made to be static types to allow for compile time data movement optimisation for compile-time embedded DSELs.

Implicit Data Movement

We may wish to introduce a way of implicitly moving data between execution contexts without the need for explicitly acquiring and releasing the data. This could be done by having the control structures such as `async` perform the put and get implicitly, though this removes any ability to create continuations on data movement operations.

Hierarchical Memory Structures

While CPUs have a single flat memory region with a single address space, most heterogeneous devices have a more complex hierarchy of memory regions each with their own distinct address spaces. Each of these memory regions have a unique access scope, semantics and latency. Some heterogeneous programming models provide a unified or shared memory address space to allow more generic programming such as OpenCL 2.x [6], HSA [7] and CUDA [8], however, this will not always result in the most efficient memory access. This can be supported either in hardware where the host CPU and remote devices share the same physical memory or software where a cross-device cache coherency system is in place, and there are various different levels at which this feature can be supported. In general, this means that pointers that are allocated in the host CPU memory region can be used directly in the memory regions of remote devices, though this sometimes requires mapping operations to be performed.

We may wish to investigate this feature further to incorporate support for this kind of systems, ensuring that the `managed_ptr` can fully utilise the memory regions on these systems.

References

[1] P0443R0 A Unified Executors Proposal for C++:
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0443r0.html

[2] SYCL 1.2 Specification:
https://www.khronos.org/registry/sycl/specs/sycl-1.2.pdf

[3] STEllAR-GROUP HPX Project:
https://github.com/STEllAR-GROUP/hpx

[4] KoKKos Project:
https://github.com/kokkos

[5] Raja Project:
http://software.llnl.gov/RAJA/

[6] OpenCL 2.2 Specification
https://www.khronos.org/registry/OpenCL/specs/opencl-2.2.pdf

[7] HSA Specification
http://www.hsafoundation.com/standards/

[8] CUDA Unified Memory
https://devblogs.nvidia.com/parallelforall/unified-memory-in-cuda-6/